```
  final case class Increase(amount: Int) extends Command
  final case class Decrease(amount: Int) extends Command

  ...
}
```

You may wonder why the protocol type is now called `Command`. This is a common name that makes it easy to see which messages should request an action from the actor. Not all messages are meant to issue commands to the actor; you see some examples in the next chapter.

NOTE You may have noticed that you can use `Increase` with a negative number to decrease the total of the wallet. To avoid this, you can use a function that ensures that you always take the absolute value of the amount when reading it or raises an error.

The wallet has an initial value of `0` and a maximum value of `2`. That's why you have `WalletState(0,2)` as a factory method in listing 2.4. The actor is created using input parameters (more on creation after the listing). Once you have created the wallet, you can send messages using its protocol. For simplicity, only the increment of the wallet is shown here.

Listing 2.4 Application that creates a wallet

```
object WalletStateApp extends App {

  val guardian: ActorSystem[WalletState.Command] =
    ActorSystem(WalletState(0, 2), "wallet-state")     ❶
  guardian ! WalletState.Increase                      ❷
  guardian ! WalletState.Increase                      ❷
  guardian ! WalletState.Increase                      ❷

}
```